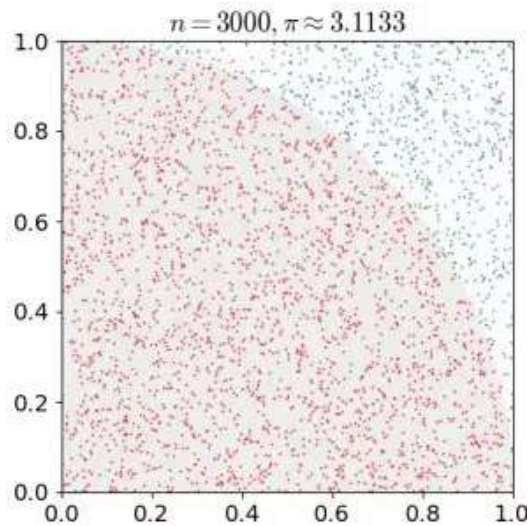


Monte Carlo & Temporal Difference Learning

Naf Guo
2026

蒙特卡洛方法



考察一个半径为 r 的圆，其面积是 πr^2

则1/4圆的面积是 $\frac{1}{4}\pi r^2$ ，对应左图粉色部分

而左图方形的面积是 r^2

现在我们在整个方形上随机放置一个个点，统计这些点有多少在粉色区间内

则其比值应该是 $\frac{1}{4}\pi$

蒙特卡洛方法就是这样大量仿真采样，然后根据仿真结果统计的方法

蒙特卡洛方法

Two types of Value-Based Methods

State Value Function:

$$V_{\pi}(s) = \mathbf{E}_{\pi}[G_t | S_t = s]$$

Value of state s

Expected return

If the agent starts at state s

And uses the policy to choose its actions for all time steps

For each state,
the state-value function outputs
the expected return
if the agent starts in that state
and then follows the policy forever after.

Deep RL Course

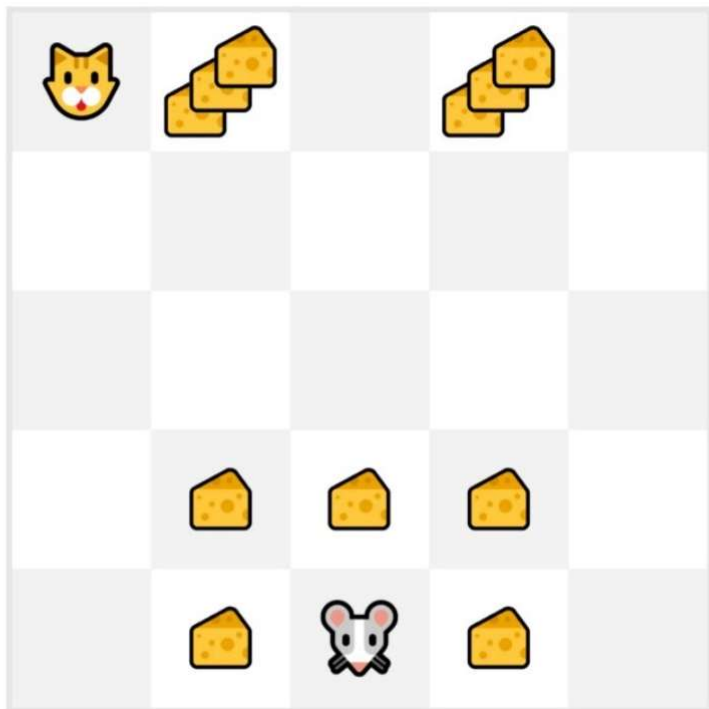
现在我们有价值函数的式子，我们要怎么用蒙特卡洛方法获得价值数值呢？

我们的做法也就是按照给定策略进行大量仿真，获得很多轨迹，之后再直接用定义去计算价值函数。

这里提到了轨迹 trajectory，就是一个序列：

$$\tau = (s_0, a_0, s_1, a_1, s_2, a_2 \dots)$$

蒙特卡洛方法

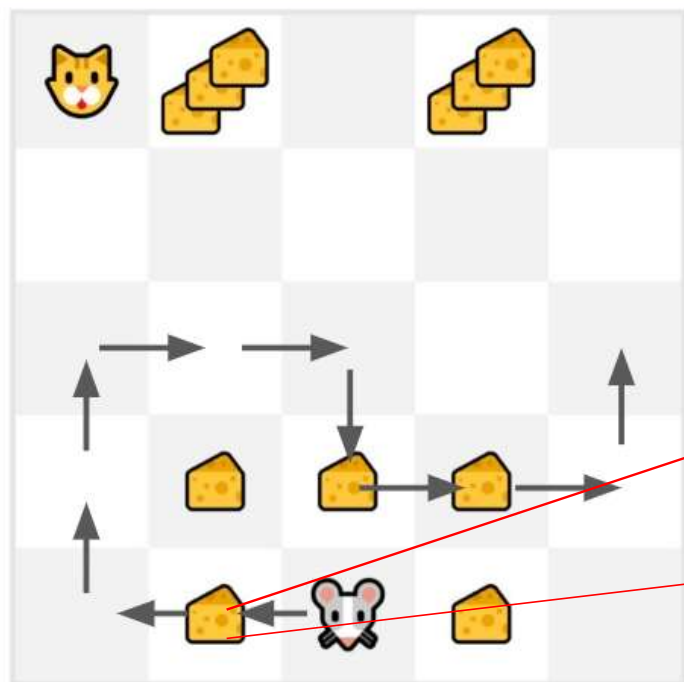


我们举一个这样的例子：

老鼠从图示位置出发，每次走一步，按照格子里的奶酪数目获得reward，达到最大步数10步，或者走到猫格子里游戏结束。

那蒙特卡洛方法就是从图示位置出发，采取某种策略如随机，走10步，获得很多条游戏轨迹。

蒙特卡洛方法



比如我们采样得到一条这样的轨迹

则轨迹总的return是 $1 + 0 + 0 + 0 + 0 + 0 + 1 + 1 + 0 + 0 = 3$

现在看老鼠出发位置左边一个格子, 在这条轨迹里, 按定义, 价值是从它出发得到的return, $0 + 0 + 0 + 0 + 0 + 1 + 1 + 0 + 0 = 2$

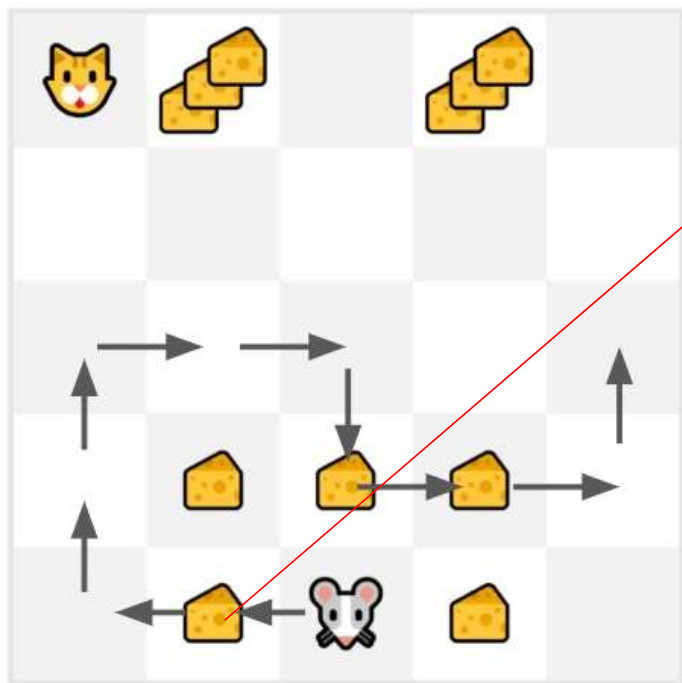
那对我采样得到的所有轨迹, 找到那些经过了这个格子的轨迹, 都去计算这个格子后续的return, 算期望就是它的价值了。

$$\underbrace{V_{\pi}(s)}_{\text{Value of state } s} = \underbrace{\mathbf{E}_{\pi}}_{\text{Expected return}} [\underbrace{G_t | S_t = s}_{\text{If the agent starts at state } s}]$$

And uses the policy to choose its actions for all time steps

For each state,
the state-value function outputs
the expected return
if the agent starts in that state
and then follows the policy forever after.

蒙特卡洛方法



对这个格子，我们刚才用蒙特卡洛方法计算得到了回报期望是 G_t ，理论上这就是这个格子的价值分数了，但我们之前对这个格子有一个价值分数估计，是 $V(S_t)$ ，我们要怎么根据 G_t 去更新得到新的价值分数呢？

一种想法是直接 G_t 替换，但这样就完全浪费了之前的估计，而且会极其受采样得到的轨迹随机性影响，造成训练不稳定

我们的做法是取一个学习率 α

$$V(S_t) \leftarrow (1 - \alpha)V(S_t) + \alpha G_t$$

$V(S_t)$ 是之前的估计，现在有一条新轨迹，按这条轨迹价值是 G_t ，我们用系数 α 取一个旧信息和新信息的折中

蒙特卡洛方法

Monte Carlo Approach:

Monte Carlo: waits until the end of the episode, then calculates G_t (return) and uses it as a target for its value or policy.

$$\underbrace{V(S_t)}_{\text{New value of state } t} \leftarrow \underbrace{V(S_t)}_{\text{Former estimation of value of state } t \text{ (= Expected return starting at that state)}} + \underbrace{\alpha}_{\text{Learning Rate}} \underbrace{[G_t - V(S_t)]}_{\text{Return at timestep } t}$$

我们每获得一条新轨迹就可以把对应所有的价值都更新一次。

如图的意思是，对 S_t 的价值，之前的估计是 $V(S_t)$ ，现在我采了一条新轨迹，这条轨迹里， S_t 对应的return是 G_t ，那我就根据这个 G_t 去更新 $V(S_t)$ ， α 是一个学习系数。

上面的式子变形一下就是 $V(S_t) \leftarrow (1 - \alpha)V(S_t) + \alpha G_t$

$V(S_t)$ 是之前的估计，现在有一条新轨迹，按这条轨迹价值是 G_t ，我们用系数 α 取一个旧信息和新信息的折中

蒙特卡洛方法

- We always start the episode **at the same starting point**.
- **The agent takes actions using the policy**. For instance, using an Epsilon Greedy Strategy, a policy that alternates between exploration (random actions) and exploitation.
- We get **the reward and the next state**.
- We terminate the episode if the cat eats the mouse or if the mouse moves > 10 steps.
- At the end of the episode, **we have a list of State, Actions, Rewards, and Next States tuples** For instance `[[State tile 3 bottom, Go Left, +1, State tile 2 bottom], [State tile 2 bottom, Go Left, +0, State tile 1 bottom]...]`
- **The agent will sum the total rewards** G_t (to see how well it did).
- It will then **update** $V(s_t)$ **based on the formula**

$$\underbrace{V(S_t)}_{\text{New value of state t}} \leftarrow \underbrace{V(S_t)}_{\text{Former estimation of value of state t (= Expected return starting at that state)}} + \underbrace{\alpha}_{\text{Learning Rate}} \underbrace{[G_t - V(S_t)]}_{\text{Return at timestep t - Former estimation of value of state t (= Expected return starting at that state)}}$$

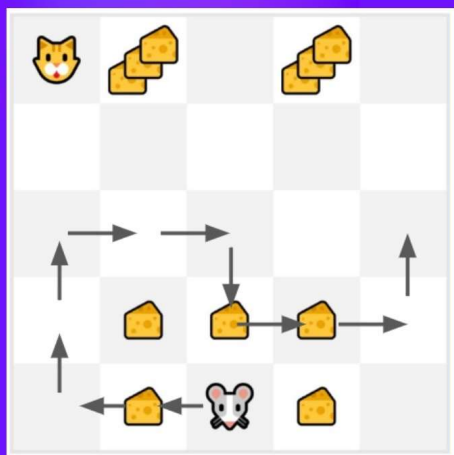
- Then **start a new game with this new knowledge**

每采一条新轨迹，就进行一次迭代

这样，每一条新轨迹在采样的时候就用到了之前的学习成果，它可能就会“更聪明”，最接近最优策略。

蒙特卡洛方法

Monte Carlo Approach:



- Calculate the return G_t .

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} \dots$$

$$G_t = 1 + 0 + 0 + 0 + 0 + 0 + 1 + 1 + 0 + 0$$

$$G_t = 3$$

- We can now update $V(S_0)$.

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$

$$\text{New } V(S_0) = V(S_0) + \text{lr} * [G_t - V(S_0)]$$

$$\text{New } V(S_0) = 0 + 0.1 * [3 - 0]$$

$$\text{New } V(S_0) = 0.3$$

最开始的时候每个状态的价值都初始化为0，采样一条轨迹以后更新路过的价值函数。

之后根据新的价值函数再采样，这样我们的小老鼠看起来就会越来越“聪明”

当然，采样的时候一般会采取 **Epsilon Greedy** 策略来平衡探索和利用。

时序差分方法

The Bellman Equation

$$V_{\pi}(s) = \mathbf{E}_{\pi} [R_{t+1} + \gamma * V_{\pi}(S_{t+1}) | S_t = s]$$

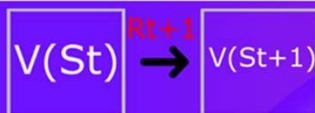
Value of
state s

Expected value of
immediate reward

+ the discounted value of
next_state

If the agent
starts at state s

And uses the policy to
choose its actions for
all time steps



$$V(S_t) = R_{t+1} + \gamma * V(S_{t+1})$$



蒙特卡洛方法需要有一条完整的轨迹才能开始更新价值函数。

那我们会想，能不能每走一步就更新一次？边采样边更新。

根据贝尔曼方程，我们注意到两个状态的价值函数是有一个关系的。

时序差分方法

TD Learning Approach:

Temporal Difference Learning: learning at each time step.

$$\underbrace{V(S_t)}_{\text{New value of state } t} \leftarrow \underbrace{V(S_t)}_{\text{Former estimation of value of state } t} + \underbrace{\alpha}_{\text{Learning Rate}} [\underbrace{R_{t+1}}_{\text{Reward}} + \underbrace{\gamma V(S_{t+1})}_{\text{Discounted value of next state}} - \underbrace{V(S_t)}_{\text{Former estimation of value of state } t}]$$

TD Target

TD 目标

我们从状态 S_t 出发走一步，获得reward R_{t+1} ，到达状态 S_{t+1} 。

按照蒙特卡洛方法，我们需要完成这条轨迹才能获得 G_t 。

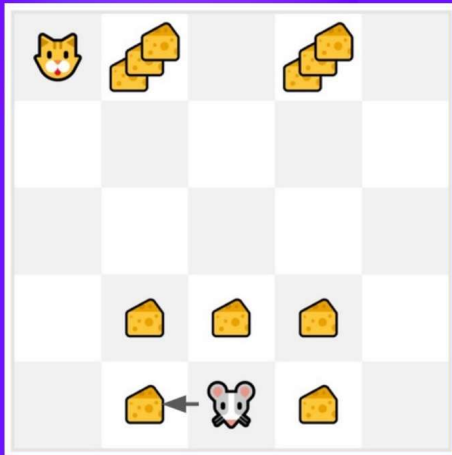
但根据贝尔曼方程，我们其实知道

$$G_t = R_{t+1} + \gamma V(S_{t+1})$$

可以直接用它们去代替完成轨迹才能有的 G_t ，这样每走一步就可以进行一次更新。

时序差分方法

TD Approach:



- We can now update $V(S_0)$:

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

New $V(S_0) = 0 + 0.1 * [1 + 1 * 0 - 0]$
The new $V(S_0) = 0.1$

So we just updated our value function for State 0.

Now we continue to interact with this environment with our updated value function.

同样地，每个位置的价值都初始化为0，之后每走一步都可以进行一次更新。